

Priorities for Software

Correctness

Performance

Usability

⇒ Once you have a minimal viable product (correctness), make it fast "enough"

⇒ Once it's fast enough, get user feedback to iterate on ease-of-use

Fast Enough

No clear-cut definition, but some factors:

- User-perceptible lag (>100 ms)
- User-annoying lag (>1000 ms)
- Common system configurations vs underpowered ones
- Time to system failure/interrupt (10 hour download, 72 hour task)

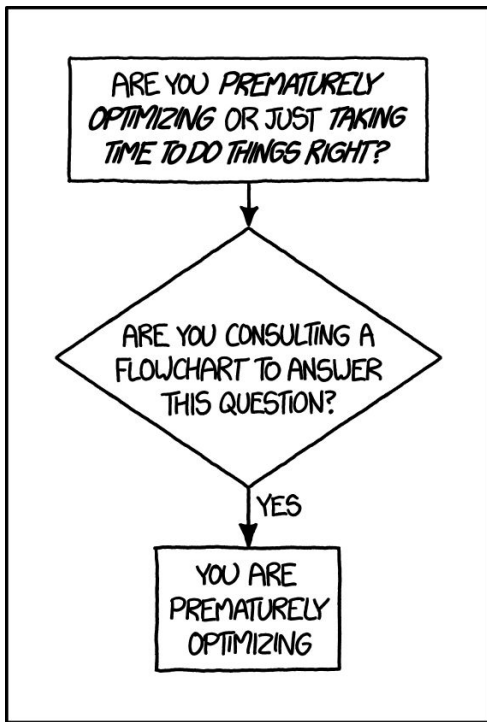
Performance Tuning

Premature optimization is doing performance tuning before you have correctness finished

- Grey areas around design decisions

Downsides

- Waste of time (bottlenecks don't end up being what you expect)
- Complicates code unnecessarily



Performance Tuned Code Gets Messy

```
// NOT THREAD SAFE. Omits a bunch of checks for the sake of
speed, relying on our specific
// calling context
public int readLine(char[] readInto) throws IOException {
    int pos = 0;
    int startChar;
    boolean omitLF = skipLF;
    for (;;) {
        if (nextChar >= nChars)
            fill();
        if (nextChar >= nChars) { /* EOF */
            return pos;
        }
        boolean eol = false;
        char c = 0;
        int i;
        /* Skip a leftover '\n', if necessary */
        if (omitLF && (cb[nextChar] == '\n'))
            nextChar++;
        skipLF = false;
        omitLF = false;
```

```
charLoop:
        for (i = nextChar; i < nChars; i++) {
            c = cb[i];
            if ((c == '\n') || (c == '\r')) {
                eol = true;
                break charLoop;
            }
        }
        startChar = nextChar;
        nextChar = i;
        if (eol) {
            System.arraycopy(cb, startChar, readInto, pos, i -
startChar);
            pos += (i - startChar);
            nextChar++;
            if (c == '\r') {
                skipLF = true;
            }
            return pos;
        }
        System.arraycopy(cb, startChar, readInto, pos, i - startChar);
        pos += (i - startChar);
    }
}
```

Performance Tuning - Why It Matters

Something that's "a little slow" on a dev machine with a fios internet connection is likely unusable on a phone over a 3G network

- Global accessibility
- Wider user base
- Build other components on top of existing software

Constant Factors vs Big-O

We're mainly focusing on constant factors here, but it's worth noting:

An algorithmic improvement is often orders of magnitude faster than tweaking constants

Changing underlying algorithms is non-trivial, though

How to Tune for Performance

Always, always, always **measure** to see where performance bottlenecks are, do not try to guess.

Real life example: a buggy cache lookup introduced a 30% slowdown to the 'save changes' pipeline (in a situation that auto-saved on keystroke)

Once you know what **method** causes a bottleneck, then investigate

- Having tightly scoped methods helps with this

Performance Issue Causes

Once you have a method that's causing problems, things to look for:

- Loops
- Network calls
- File system writes
- Locking

Performance Issue Causes

Ex: Loops

```
public void doAThing() {  
    int start = 0;  
    for (int i = 0; i < 1_000_000; i++) {  
        double multiple = Math.pow(3,9);  
        start *= multiple;  
    }  
    System.out.println(start);  
}
```

Factor out common logic from **inner loops**

Performance Issue Causes

Ex: Loops

```
public void doAThing() {  
    int start = 0;  
    double multiple = Math.pow(3,9);  
  
    for (int i = 0; i < 1_000_000; i++) {  
        start *= multiple;  
    }  
    System.out.println(start);  
}
```

Better: multiple only gets calculated once

Performance Issue Causes

Ex: Network Calls

```
public List<String> loadDatabaseTable(Database db, String table) {  
    List<String> loadedData = new ArrayList<>();  
  
    int numRows = db.getSizeOfTable(table);  
    for (int i = 0; i < numRows; i++) {  
        loadedData.add(db.getRow(table, i));  
    }  
    return loadedData;  
}
```

Cross-network APIs (should) provide a **batch/bulk** read option, use that for large amounts of data

Performance Issue Causes

Ex: Network Calls

```
public List<String> loadDatabaseTable(Database db, String table) {  
    int numRows = db.getSizeOfTable(table);  
    List<String> loadedData = new ArrayList<>(numRows);  
  
    Iterator<String> rowCursor = db.getRows(table);  
    while (rowCursor.hasNext()) {  
        String row = rowCursor.next();  
        loadedData.add(row);  
    }  
    return loadedData;  
}
```

Better: only 1 + #pages network calls, plus bonus avoid resizing the list

Performance Issue Causes

Ex: File System Writes

```
public void writeSomeData(FileWriter out) throws IOException {  
    for (int i = 0; i < 10; i++) {  
        String largeDataBlob = expensiveCalculation(i);  
        out.write(largeDataBlob);  
    }  
}
```

Several solutions:

- Producer/Consumer pattern
- Buffering (some built in, but might need more)

Performance Issue Causes

Ex: File System Writes

```
public void writeSomeData(BufferedWriter out) throws IOException {  
    for (int i = 0; i < 10; i++) {  
        String largeDataBlob = expensiveCalculation(i);  
        out.write(largeDataBlob);  
    }  
}
```

Better: Default buffer size for BufferedWriter is 8k, vs 1k for FileWriter

⇒ Often, performance tuning means poking under the hood for library classes

Performance Issue Causes

Ex: Locking

If threads spend
a lot of time in
blocked wait,
revisit ways to
improve
parallelism

```
private int currentCount = 0;
public int doARead(Semaphore lock) throws InterruptedException {
    lock.acquire();
    try {
        return currentCount;
    } finally {
        lock.release();
    }
}
public void doAWrite(Semaphore lock) throws InterruptedException {
    lock.acquire();
    try {
        currentCount++;
    } finally {
        lock.release();
    }
}
```

Performance Issue Causes

Ex: Locking

Better: Use a
ReadWriteLock
to allow multiple
reads at once

```
private int currentCount = 0;
public int doARead(ReadWriteLock lock) throws InterruptedException {
    lock.readLock().lock();
    try {
        return currentCount;
    } finally {
        lock.readLock().unlock();
    }
}
public void doAWrite(ReadWriteLock lock) throws InterruptedException {
    lock.writeLock().lock();
    try {
        currentCount++;
    } finally {
        lock.writeLock().unlock();
    }
}
```


Your Turn!

1. Clone the PerformanceTuningExercises repo from the link on Brightspace
2. Find the `thisMethodIsTooSlow` method in the `exercise1` package
3. Change the method in a way that's likely to make it run faster (we'll cover how to measure that in a bit)

Performance Testing

Benchmarks serve as regression tests for performance

- Standard set of operations covering most major behavior
- Known amount of time to run
- A small standard deviation in the time it takes to run

To run usefully:

- Same/similar hardware or environment (virtualization often doesn't work)
- Ideally, run many times and averaged (micro benchmarks)

Performance Testing

Always benchmark before and after **each** change that you make to address a performance issue

Ex:

commit 1 improves performance by 10%

commit 2 degrades performance by 5%

Combined, there's still a performance improvement

Benchmark Tests

Need to compare a **baseline** to a changed valued

Hardcode (or store in a config file) a known baseline value

- similar to 'golden tests' for UI testing
- should fail for **both** sufficiently large improvements and regressions

For end-to-end tests (or localized testing), often possible to run on two different versions within the test and directly compare

- Greater resiliency to changing hardware
- Auto-updates to new good values, so no need to fail for improvements

Benchmark Tests

How big should a change be before getting flagged?

Depends on the variability within the test (avoid false positives)

Larger values risk creeping slowdowns over time

10% is a common value

- combined with comparing to the previous major release as well as previous commits to minimize incremental slowdowns

Performance Trade Offs

Make one part faster in exchange for another part becoming slower.

Ex: Binary Search Tree vs Array List

- Insert is slower
- Lookup is faster

Performance Trade Offs

Common pitfall: "micro" benchmarks that just test constant factors for very small cases

Make sure you aren't making an already-fast task faster at the expense of a slower task becoming slower

Do not measure performance in nanoseconds - nanos make it hard to tell if you've got a micro benchmark.

- For java, use `System.currentTimeMillis()`
- Benchmarks should take 8-60 ms (still fast, slow enough to give real data)

Performance Trade Offs

Document any trade-offs

- Typically in comments in the code
- For major design decisions that impact how the system is used, document in the README

Ex: 'Using bit-packing here to allow fully in-memory lookups of data' is a good class comment

'Requires running with a jvm with a 32GB heap' should go in the README

Example Benchmark Test

To the IDE!

Measuring Program Performance

Ways to measure performance:

- YourKit/instrumentation (YourKit is sadly no longer free)
- Timing logging wrappers
- System-level utilities (top, iostat, ping/netstat)

When all else fails: kill -3 produces thread dumps and can be called repeatedly

Instrumentation Tools

YourKit - if \$ is involved, use this, it's worth it

JConsole

- a better UI than kill -3 (and works on Windows)
- supports remote debugging
- Building block for many frameworks
- Included with the JDK

Instrumentation Tools

JConsole limitations

- Requires manual steps
- Imprecise

⇒ This is a tool to give you a rough idea where to start looking

Do **not** try to replace benchmarks with JConsole

JConsole

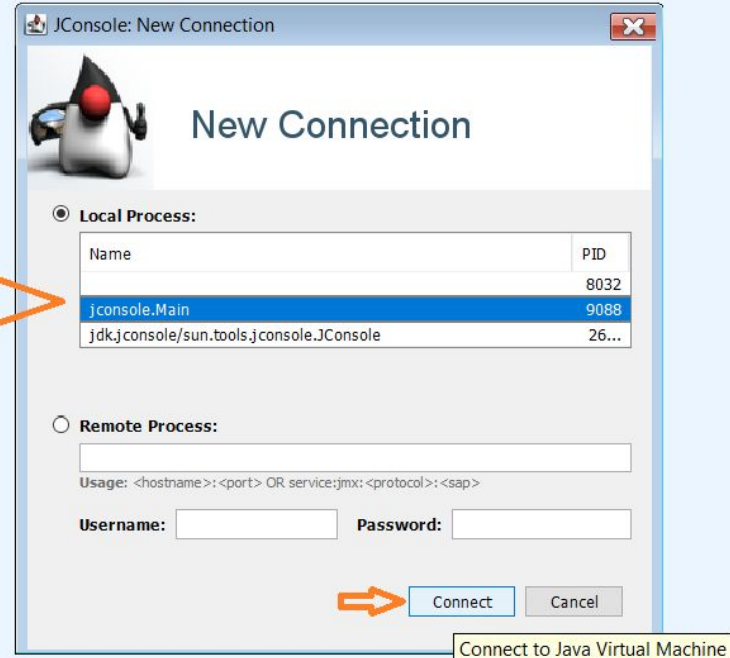
Executable (called 'jconsole') located in <path to jdk>/bin

 javafx_font.dll	2/3/2024 1:16 PM	Application extens...	65 KB
 javafx_iio.dll	2/3/2024 1:16 PM	Application extens...	158 KB
 javajpeg.dll	2/3/2024 1:16 PM	Application extens...	165 KB
 javap	2/3/2024 1:16 PM	Application	13 KB
 javaw	2/3/2024 1:16 PM	Application	39 KB
 jawt.dll	2/3/2024 1:16 PM	Application extens...	11 KB
 jcmd	2/3/2024 1:16 PM	Application	13 KB
 jconsole	2/3/2024 1:16 PM	Application	13 KB
 jdb	2/3/2024 1:16 PM	Application	13 KB
 jdepscan	2/3/2024 1:16 PM	Application	13 KB
 jdeps	2/3/2024 1:16 PM	Application	13 KB

JConsole

For local testing, often helpful to put a breakpoint at the beginning of main until you have JConsole connected

Name of your
main class

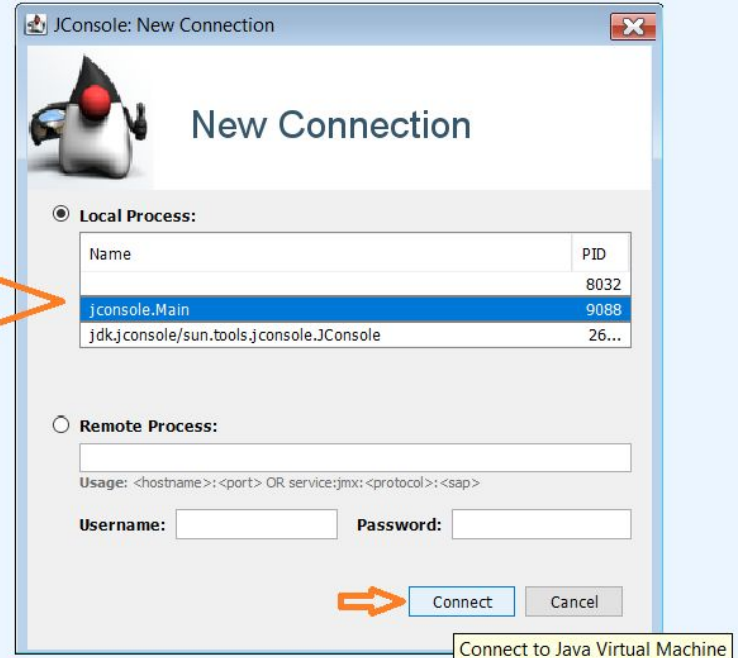


JConsole

For local testing, often helpful to put a breakpoint at the beginning of main until you have JConsole connected

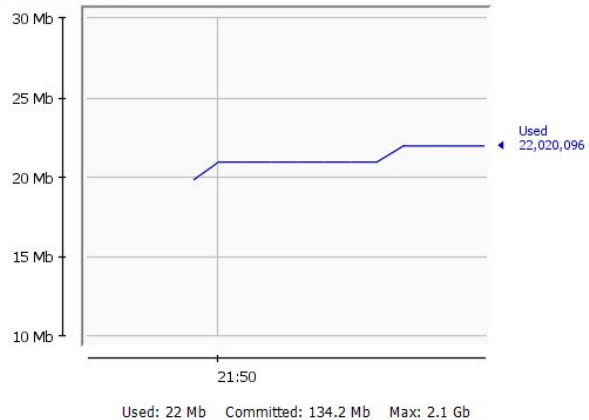
If it complains about no SSL, that's ok, just use an insecure connection

Name of your
main class

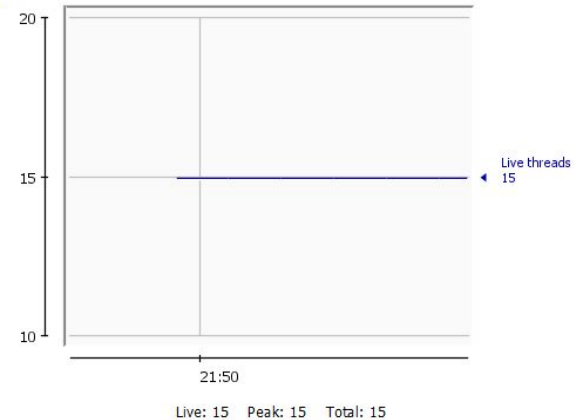


Time Range: All

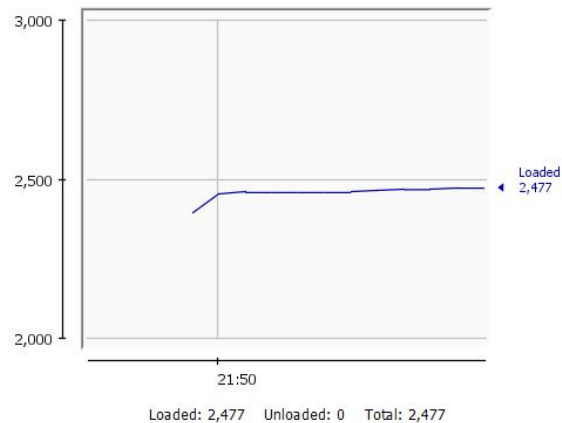
Heap Memory Usage



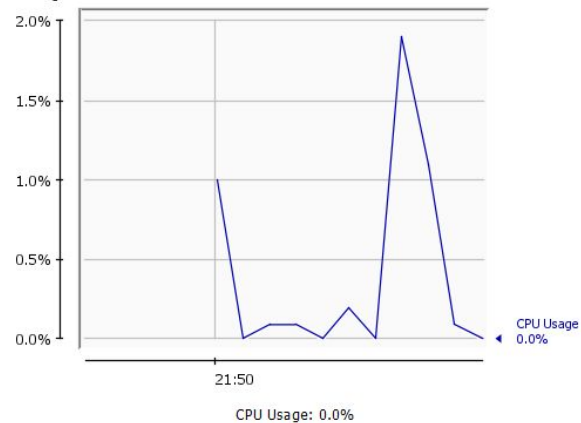
Threads



Classes

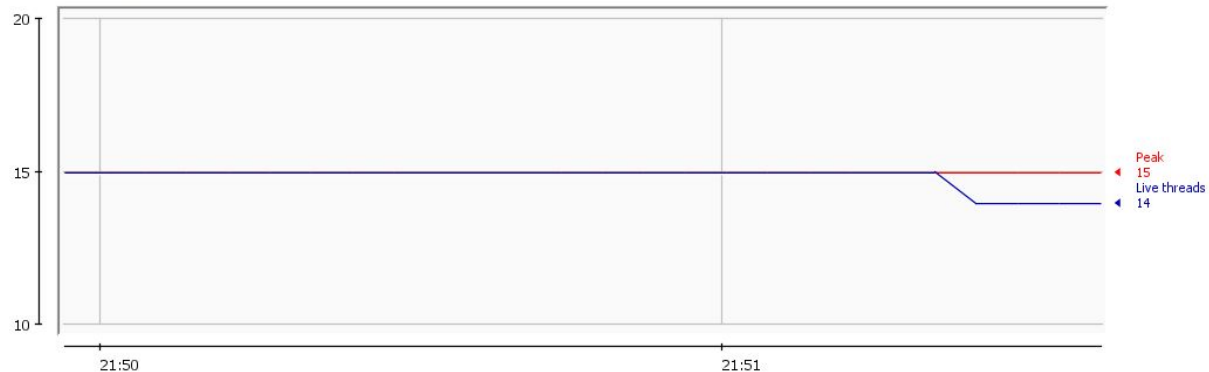


CPU Usage



Time Range: All

Number of Threads



Threads

RMI TCP Connection(3)-127.0.0.1
RMI TCP Connection(2)-127.0.0.1
JMX server connection timeout 22
RMI Scheduler(0)
RMI TCP Connection(1)-127.0.0.1
RMI TCP Accept-0
RMI TCP Accept-10001
RMI TCP Accept-0
Common-Cleaner
Attach Listener
Signal Dispatcher
Finalizer
Reference Handler
main

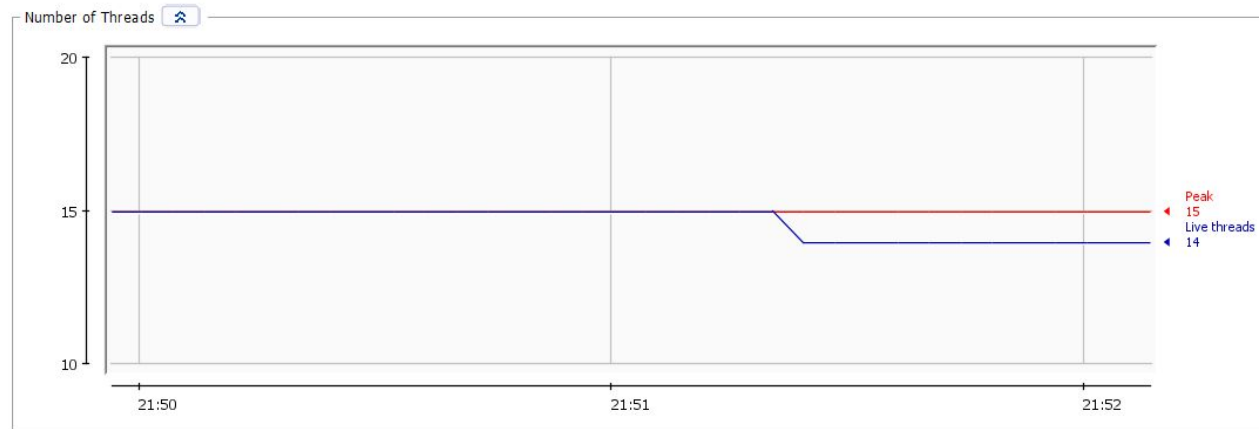
Name: main
State: RUNNABLE
Total blocked: 0 Total waited: 0

Stack trace:
app//jconsole.Main.main(Main.java:13)

Filter

Detect Deadlock

Time Range: All



Threads

RMI TCP Connection(3)-127.0.0.1
RMI TCP Connection(2)-127.0.0.1
JMX server connection timeout 22
RMI Scheduler(0)
RMI TCP Connection(1)-127.0.0.1
RMI TCP Accept-0
RMI TCP Accept-10001
RMI TCP Accept-0
Common-Cleaner
Attach Listener
Signal Dispatcher
Finalizer
Reference Handler
main

Name: main

State: TIMED_WAITING

Total blocked: 0 Total waited: 444

Stack trace:

java.base@11.0.22/java.lang.Thread.sleep(Native Method)

app//exercisel.Exercisel.thisMethodIsTooSlowOriginal(Exercisel.java:34)

app//jconsole.Main.main(Main.java:17)

Filter

Detect Deadlock

Your Turn!

1. Start with the code in the exercise2 package
2. Don't look at the code for MysteryClass yet
3. Use JConsole to find the method call(s)/section of code in PerformanceTuning.main that's causing a bottleneck

Timing Logging

Pros:

- Easier to investigate after the fact
- Useful when no remote debugging is possible
- Can provide arbitrary details

Cons:

- You have to build it or buy it
- Be careful the logging itself doesn't cause slowdowns

Timing Logging

Create a wrapper around the relevant code

Based on some condition, record stats on that code

Common conditions:

- always
- when 'debug' logging is enabled
- when a ThreadLocal flag is set

ThreadLocal

Useful type for multithreaded situations

Under the hood, it's a Map with Thread objects as keys

Can be used like a “global stack” - it's not a local variable or parameter, but also (effectively) not shared between threads

Timing Logging

Create a wrapper around the relevant code

Based on some condition, record stats on that code

Common stats:

- how often it's called
- the immediate caller (stack trace)
- how long it runs

Timing Logging

Periodically, record the timing logging somewhere

- central monitoring server
- file system

Timing Logging Example

To the IDE!

Performance Tuning: Caching

Pros:

- Avoid doing an expensive operation (network call, file system read, large computation) multiple times

Cons:

- Lots of memory
- Cache invalidation when data changes

Performance Tuning: Caching

`Suppliers.memoize()` - since `grpc` pulls in `Guava`, this is already on your classpath

```
Ex: Supplier<Integer> numRows = Suppliers.memoize(() ->
loadNumRowsFromDb());
```

`CacheBuilder` - more options for complex caches

Most common: `maximumSize`, `expireAfterWrite`, `expireAfterAccess`

Other less common features:

- use a "weight" instead of a count (`maximumWeight`, `Weigher`)
- `weakKeys`, `softValues` (less useful for classic caching)

Let's Review

- When in the development cycle should you focus on performance tuning
- Some factors to consider when deciding if a system needs further performance improvements
- Why you should base performance tuning on measurements
- What is a benchmark, and how would you create a good one
- Common sources of performance bottlenecks
- Common tools to detect a bottleneck